ARQUITECTURA Y SISTEMAS OPERATIVOS

CLASE 2

Introducción a los Sistemas Operativos

2026

Bibliografía

- Operating System Concepts. By Abraham, Silberschatz.
  - Capitulo I
  - Capitulo II
- Modern Operating Systems. By Andrew S. Tanenbaum.
  - Capitulo I

Introducción a los Sistemas Operativos

En esta clase vamos a responder 4 preguntas:

- ¿Qué es un sistema operativo?
- ¿Qué recursos administra?
- ¿Qué abstracciones ofrece?
- ¿Cómo mantiene control y eficiencia?

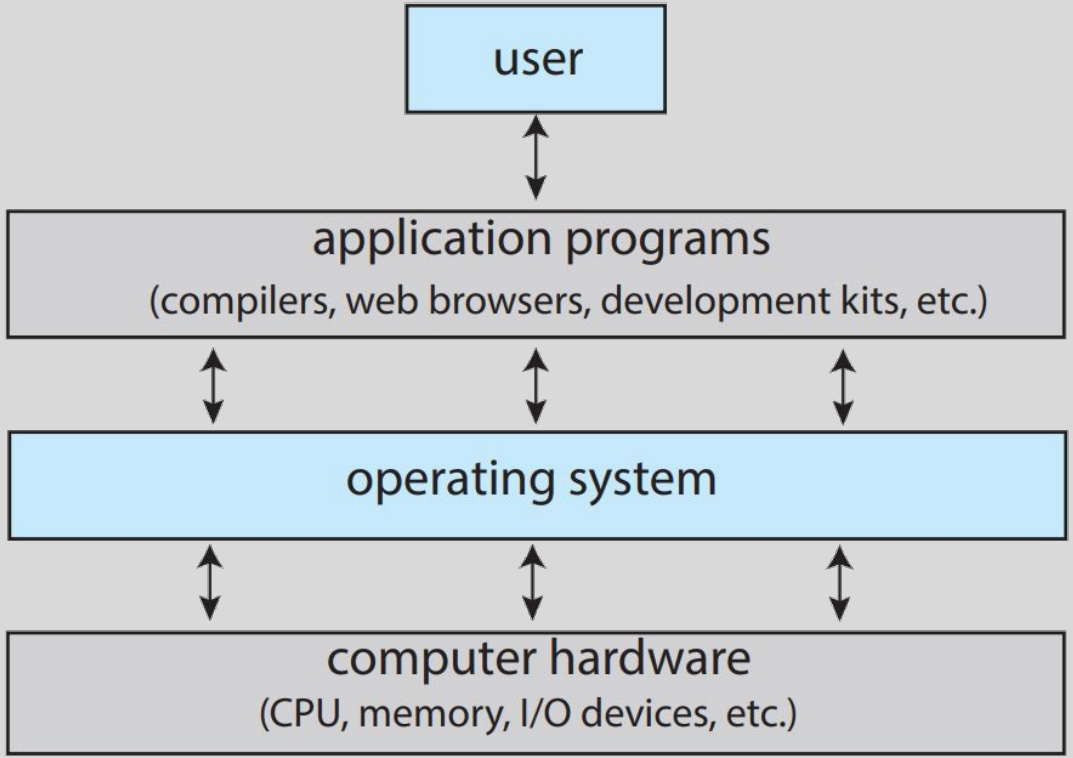
2026

¿Qué es un Sistema Operativo?

Un sistema operativo es el software que:

- Gestiona o administra el hardware
- Proporciona la base para los programas de aplicación.
- Actúa como intermediario entre usuarios, aplicaciones y hardware.

Una responsabilidad fundamental de un sistema operativo es asignar estos recursos a los programas.



¿Qué hace un Sistema Operativo?

Rol general del sistema operativo

- Coordina el uso de los recursos del sistema.
- Oculta parte de la complejidad del hardware.
- Ofrece servicios a programas y usuarios.
- Permite que múltiples actividades convivan de manera ordenada.

Dos perspectivas del sistema operativo

- Desde el usuario
  - Facilidad de uso
  - Buena respuesta
  - Entorno estable y seguro

- Desde el sistema
  - Asignación de recursos
  - Eficiencia
  - Control
  - Uso compartido ordenado

¿Por qué existe un Sistema Operativo?

¿Por qué no alcanza con el hardware?

- El hardware por sí solo no ofrece un entorno cómodo para desarrollar y ejecutar programas.
- Los programas necesitan CPU, memoria, almacenamiento y dispositivos de entrada/salida.
- Hace falta un componente central que coordine ese uso.

Ese componente es el sistema operativo.

Definición operativa: kernel y sistema

Definición operativa

- El sistema operativo es el **software** que controla el funcionamiento general del sistema.
- Su núcleo se denomina kernel.

Debemos distinguir

- **Kernel**: parte central del sistema operativo
- **Programas del sistema**: utilidades y herramientas
- **Aplicaciones**: software orientado al usuario

¿Por qué estudiar sistemas operativos?

Porque permite entender

- cómo se ejecutan realmente los programas,
- cómo se administran CPU, memoria y E/S,
- cómo se protegen los recursos,
- y cómo se relacionan software y hardware.

¿Qué administra el sistema operativo?

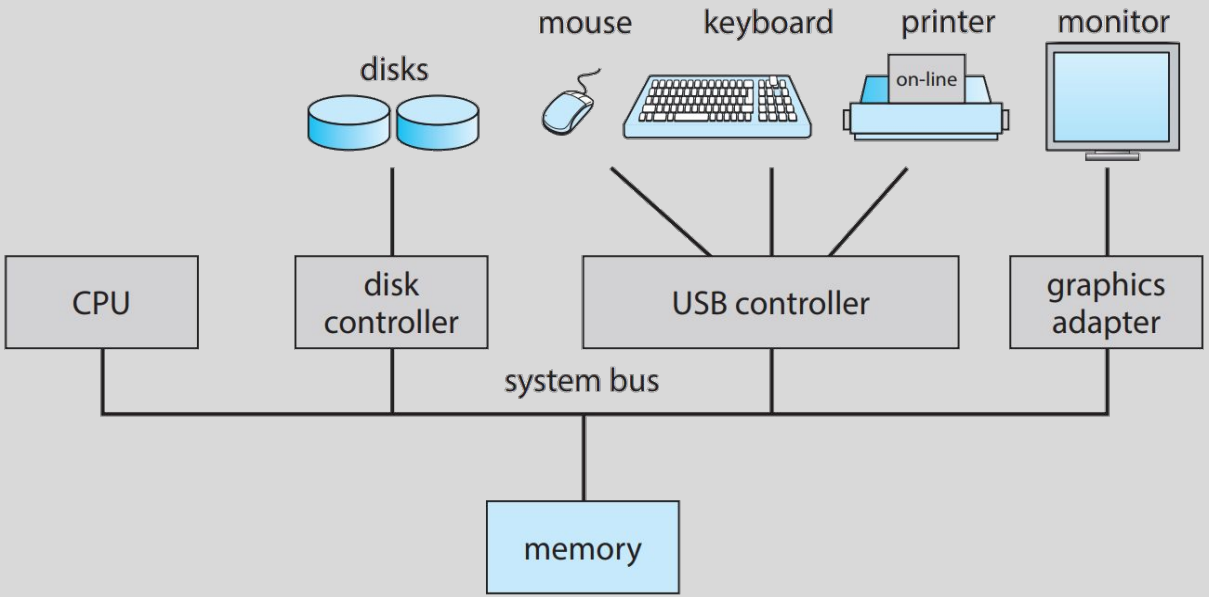
- **Recursos administrados por el SO**
  - CPU
  - Memoria
  - Dispositivos de entrada/salida
  - Almacenamiento
  - Acceso concurrente y seguro a esos recursos

Organización general de una computadora

Componentes principales:

- CPU
- Memoria principal
- Controladores de dispositivos
- Dispositivos de entrada/salida
- Almacenamiento secundario

Idea: El sistema operativo **coordina** la interacción entre estos componentes.



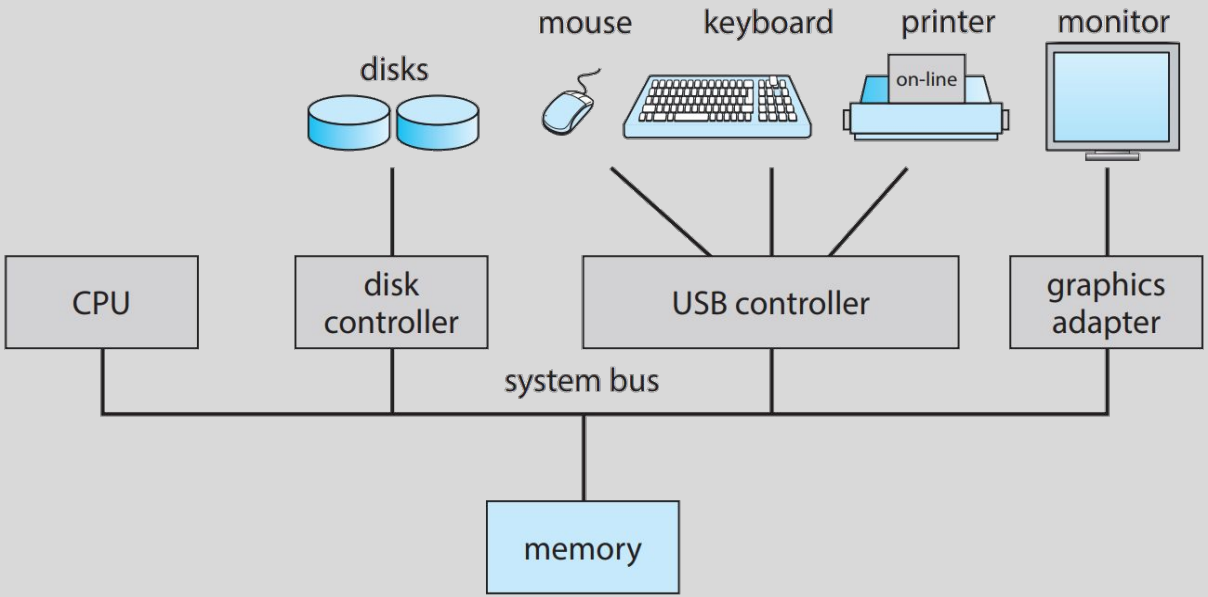
Dispositivos, controladores y drivers

Los dispositivos son heterogéneos:

- No todos funcionan de la misma manera.
- Cada uno tiene características particulares.

Para manejarlos se utilizan:

- **Device controller**: componente de hardware que controla un dispositivo
- **Device driver**: software que permite al SO comunicarse con ese controlador



Controller vs Driver

Controller vs Driver

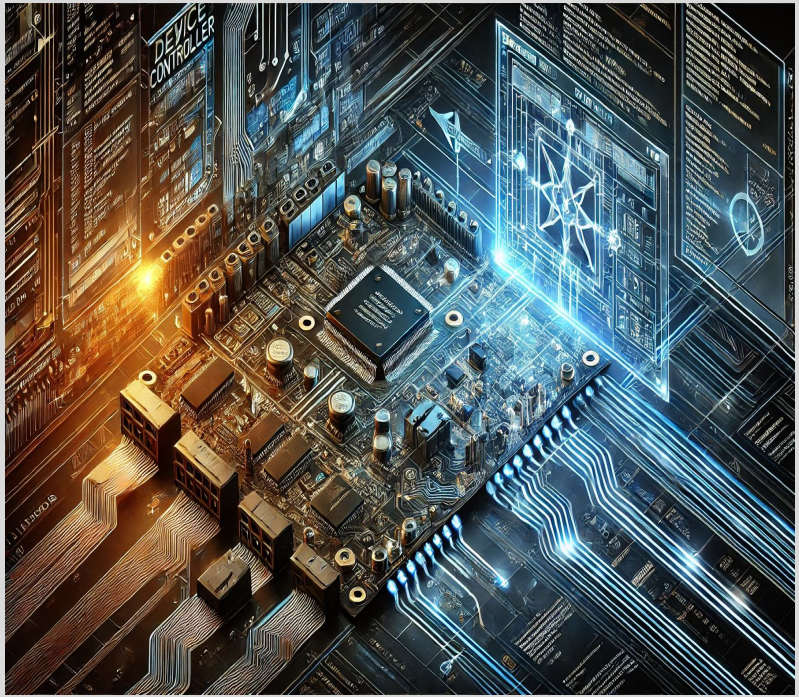
Controller:

- Es hardware
- Controla el dispositivo
- Está asociado físicamente al equipo

Driver:

- Es software
- Permite al SO usar el dispositivo
- Forma parte del software del sistema

Ambos cooperan para integrar el dispositivo al sistema operativo.



Abstracciones que ofrece el sistema operativo

El SO no expone directamente todo el hardware.

Lo que hace es ofrecer **abstracciones** que simplifican el uso del sistema.

- Ejemplos
    - Proceso
    - Archivo
    - Directorio
    - Interfaces de usuario
    - Mecanismos de protección

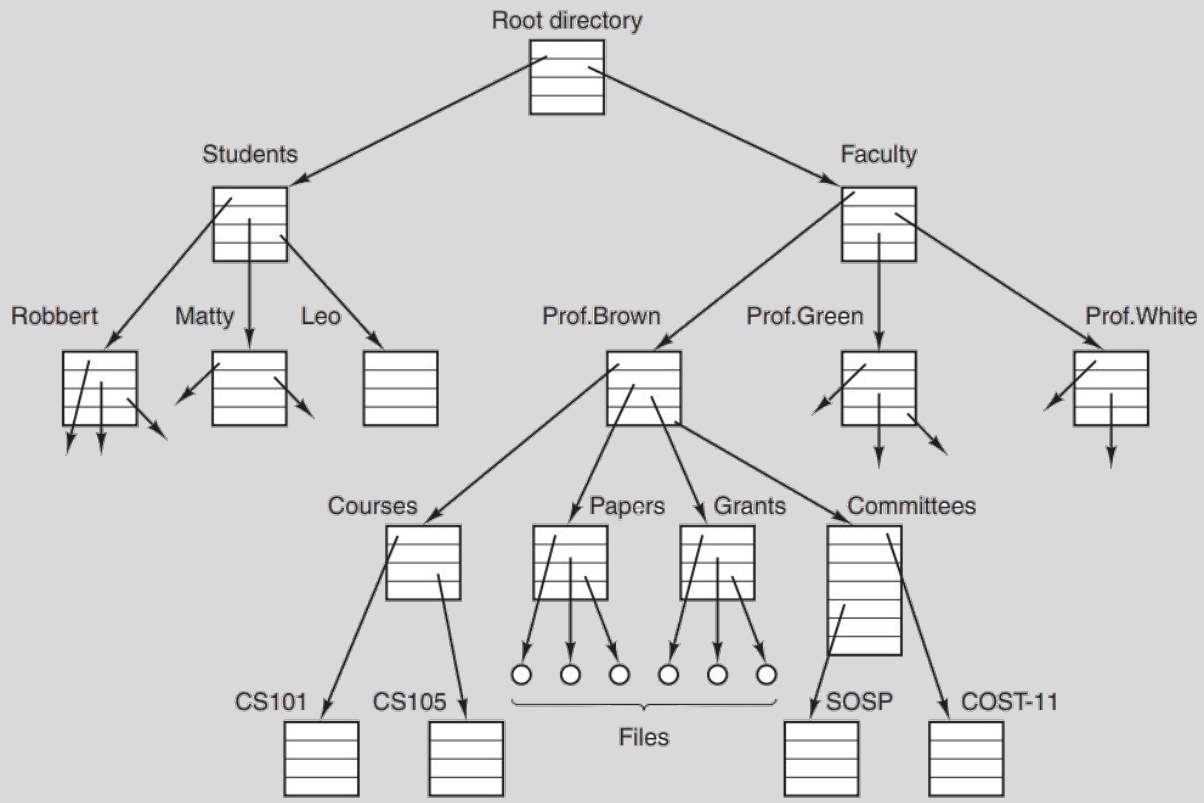
Introducción al concepto de proceso

- **Proceso**
  - Un proceso es un programa en ejecución.
- Diferencia importante
  - **Programa:** entidad pasiva
  - **Proceso:** entidad activa
- Para ejecutarse, un **proceso** necesita
  - CPU
  - Memoria
  - Archivos
  - Operaciones de E/S

Espacio de direcciones: idea general

Espacio de direcciones

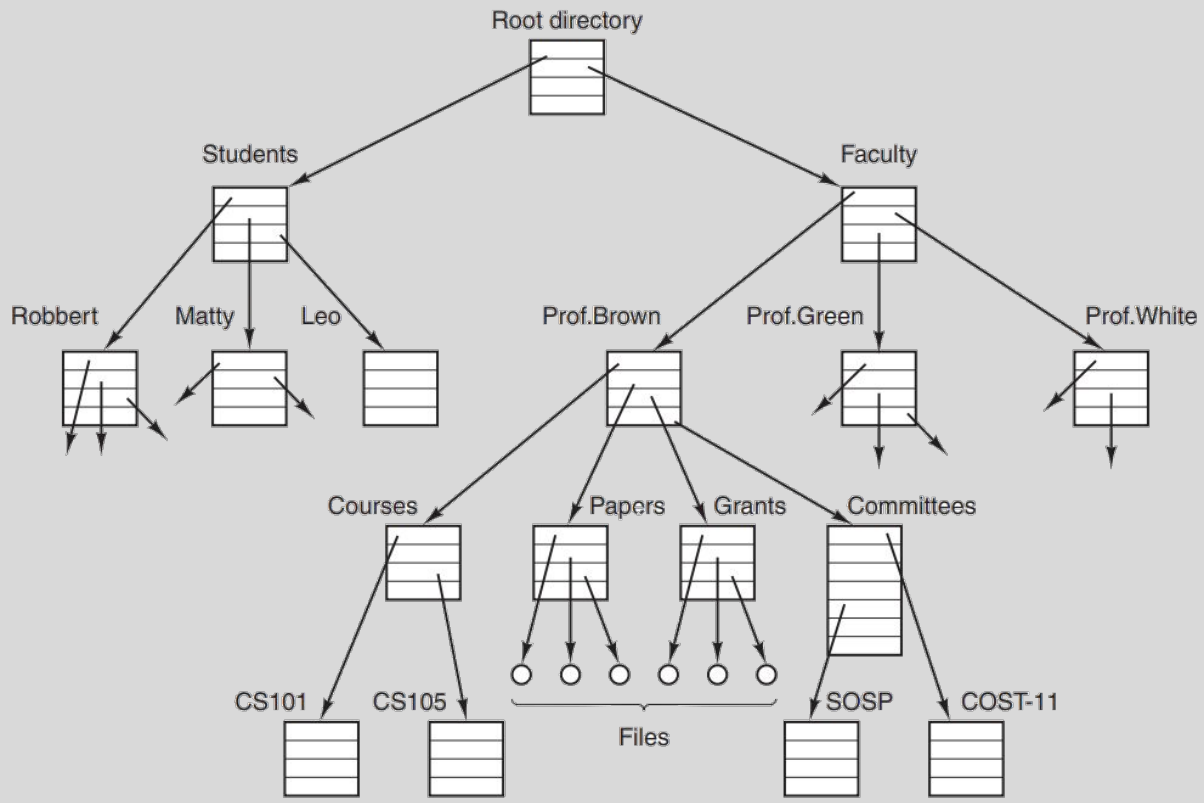
- Cada proceso se ejecuta dentro de un espacio de memoria controlado por el sistema.
- Esto ayuda a mantener orden, aislamiento y protección.
- También permite que varios programas convivan en una misma computadora.



Archivo como abstracción

Archivo

- Es una abstracción lógica para almacenar información.
- Permite trabajar con programas y datos sin depender directamente del formato físico del dispositivo.
- El sistema operativo organiza y administra esos archivos.



Ejemplo. El *path* para el archivo “CS101” es:
/Faculty/Prof.Brown/Courses/CS101

Directorios y rutas

Directorios

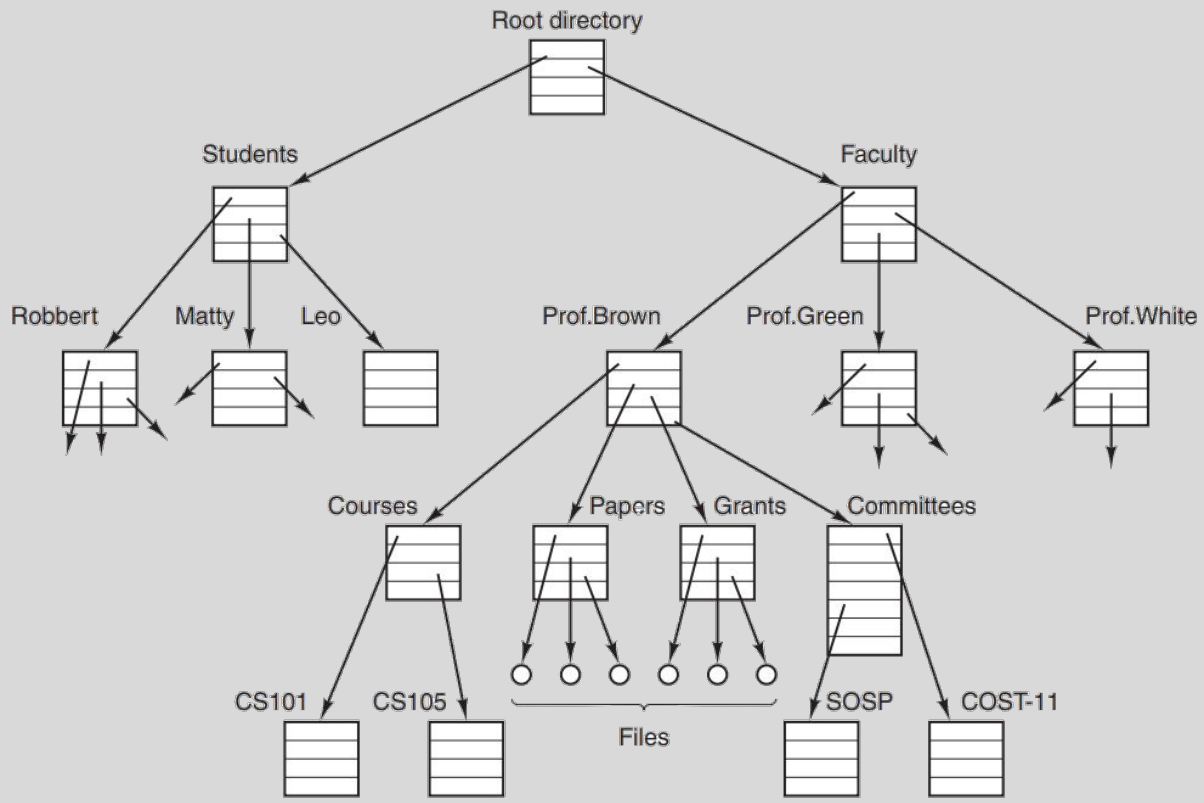
- Organizan los archivos dentro del sistema.

Rutas o paths

- Permiten ubicar archivos y directorios.

Ejemplo:

```
/home/usuario/documentos/
apunte.txt
```



Apertura de archivos y permisos

Antes de usar un archivo

- El sistema operativo verifica si el acceso está permitido.
- Si el acceso es válido, habilita las operaciones correspondientes.

Ejemplos de permisos

- Lectura
- Escritura
- Ejecución

Entrada / salida como subsistema

Subsistema de E/S

- El sistema operativo administra la entrada y salida mediante un subsistema específico.
- Parte del software de E/S es general.
- Otra parte depende del dispositivo utilizado.

Objetivo

- Desacoplar a los programas de los detalles del hardware.

Protección del sistema

Protección

- El SO debe proteger información y recursos.
- Debe controlar quién puede hacer qué.
- También debe evitar usos incorrectos o peligrosos del sistema.

Se protege, por ejemplo

- Archivos
- Memoria
- Dispositivos
- Tiempo de CPU

La shell como ejemplo de interfaz

Shell

- Es una interfaz entre el usuario y el sistema operativo.
- Permite ingresar comandos y ejecutar programas.
- No es el kernel, pero utiliza servicios del sistema operativo de forma intensiva.

Ejemplo general

- **Usuario → comando → shell → sistema operativo**

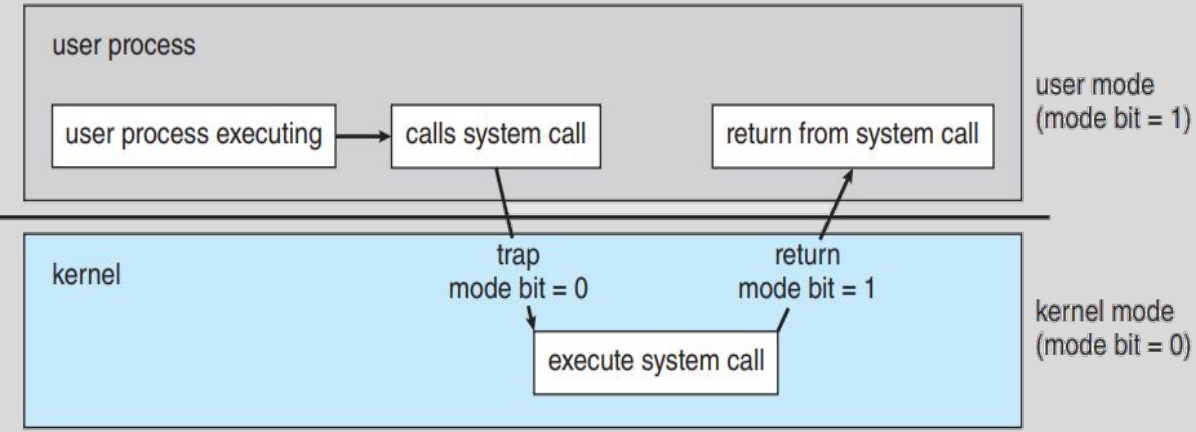
¿Cómo mantiene el control el sistema operativo?

Pregunta clave

- Si los programas usan el hardware, **¿cómo evita el SO perder el control?**

Mecanismos principales

- Modos de ejecución
- System calls
- Temporizador
- Interrupciones



Dual mode: modo usuario y modo kernel

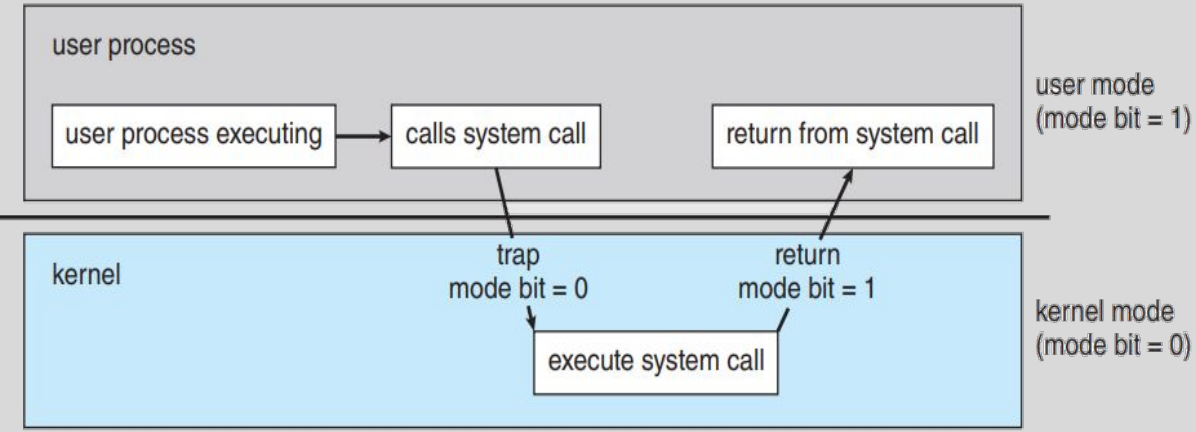
Dual mode

El hardware distingue al menos dos modos de ejecución:

- Modo usuario
- Modo kernel

Idea central:

- No todas las instrucciones pueden ejecutarse desde cualquier programa.



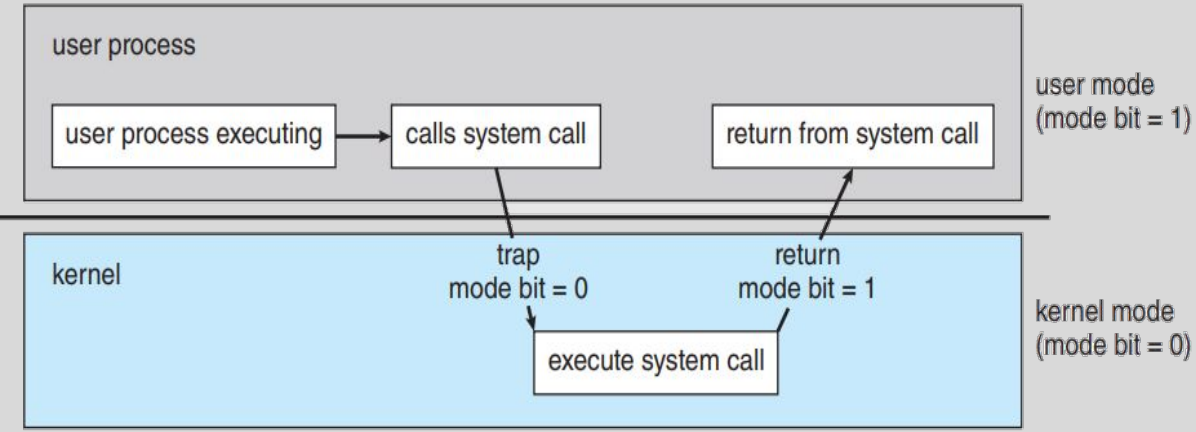
Instrucciones privilegiadas

Son operaciones que no deben quedar disponibles para cualquier programa.

Ejemplos

- Acceso directo a dispositivos
- Control de memoria
- Manejo de interrupciones
- Operaciones críticas sobre CPU

Solo pueden ejecutarse en modo kernel.



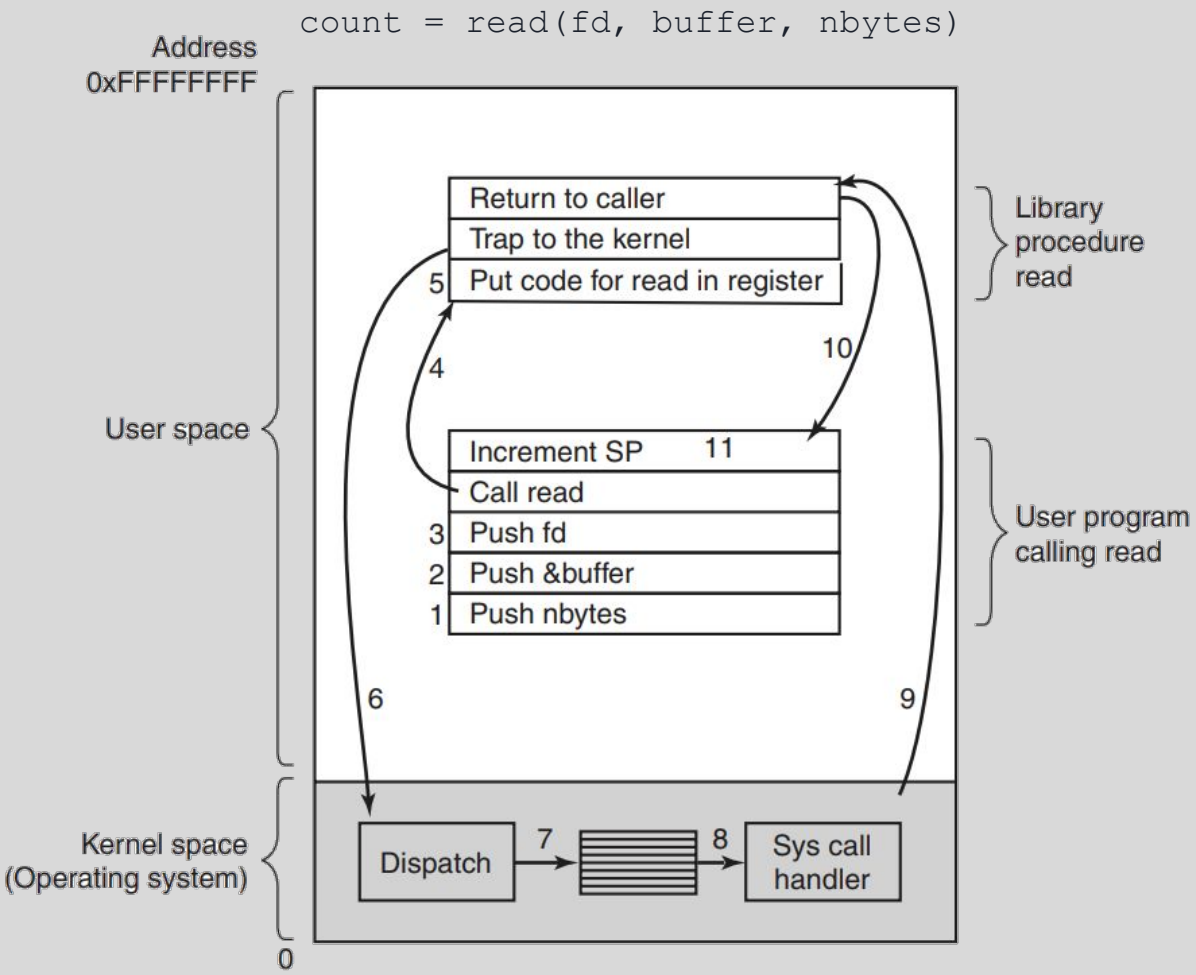
System calls: pedir servicios al SO

System call

- Es el mecanismo mediante el cual un programa de usuario solicita un servicio al sistema operativo de forma controlada.

Ejemplos:

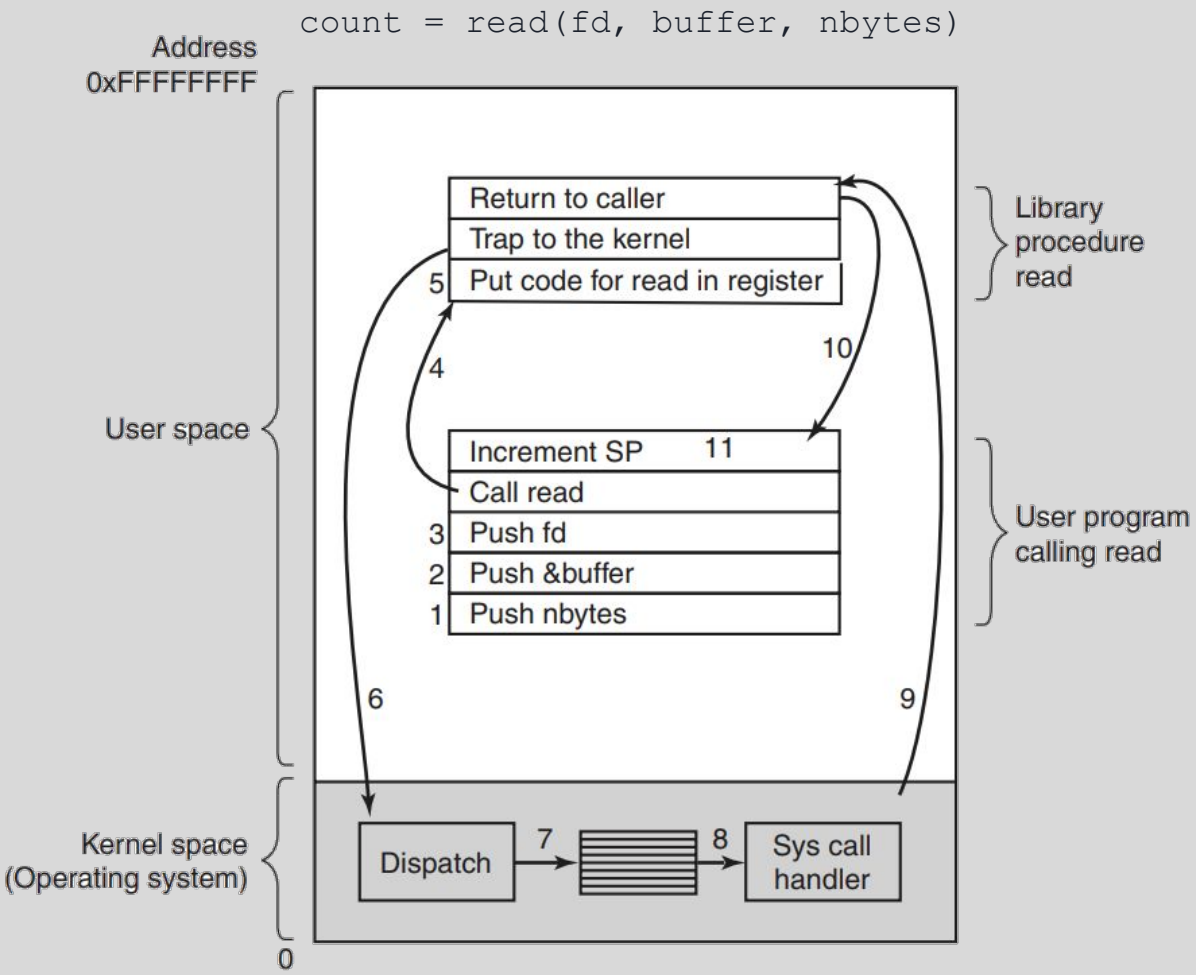
- Abrir un archivo
- Leer información
- Crear un proceso
- Reservar memoria



Flujo simplificado de una system call

Secuencia general

1. El programa solicita un servicio.
2. Se produce una transición controlada al kernel.
3. El sistema operativo realiza la operación.
4. Se devuelve un resultado al programa.
5. La ejecución continúa.



Trap o transición controlada

Entrada controlada al kernel

- Un programa no ingresa libremente al kernel.
- La transición ocurre mediante un mecanismo controlado por hardware y sistema operativo.
- Esto protege al sistema y ordena la ejecución.

Timer: cómo recupera el control el SO

Temporizador

- Permite que el sistema operativo recupere el control de manera periódica.
- Evita que un proceso monopolice la CPU.
- Hace posible una administración más justa y segura del procesador.

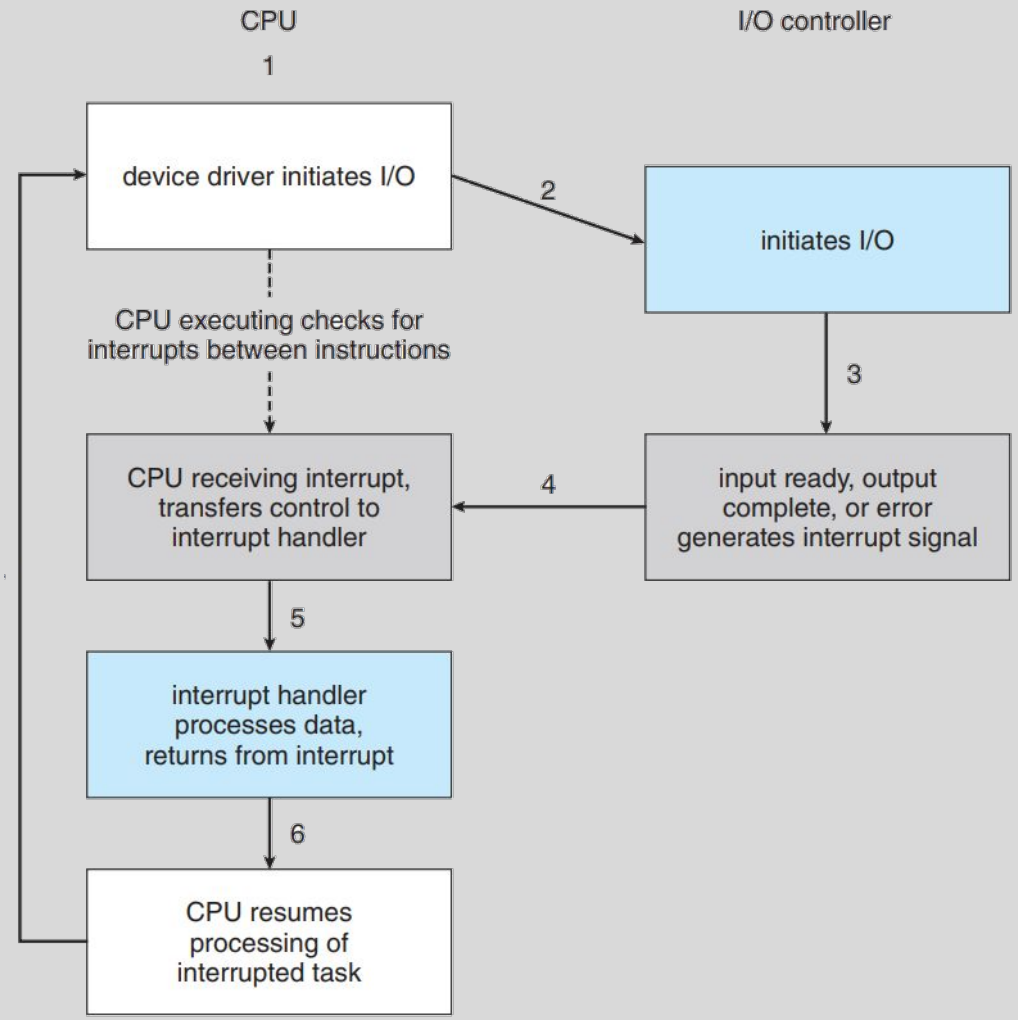
Interrupciones: eventos que exigen atención

Interrupción

- Es una señal que indica que ocurrió un evento que requiere atención del sistema.

Ejemplos

- Terminó una operación de E/S.
- Venció el timer.
- Ocurrió una condición excepcional.



Qué pasa cuando ocurre una interrupción

Secuencia general

1. Se interrumpe el flujo actual.
2. Se guarda el contexto necesario.
3. Se ejecuta la rutina correspondiente.
4. Luego se retoma la ejecución.

Idea clave

- Las interrupciones permiten reaccionar a eventos sin esperar continuamente.

Síntesis parcial

Hasta acá vimos

- Qué es el sistema operativo
- Qué abstrae
- Cómo protege el sistema
- Y cómo mantiene el control.

Ahora vemos cómo también busca eficiencia y uso compartido.

Estructura de almacenamiento

Memoria principal

- Rápida
- Limitada
- Volátil

Almacenamiento secundario

- Más lento
- Persistente
- Mayor capacidad

El sistema operativo es el encargado de coordinar ambos niveles.

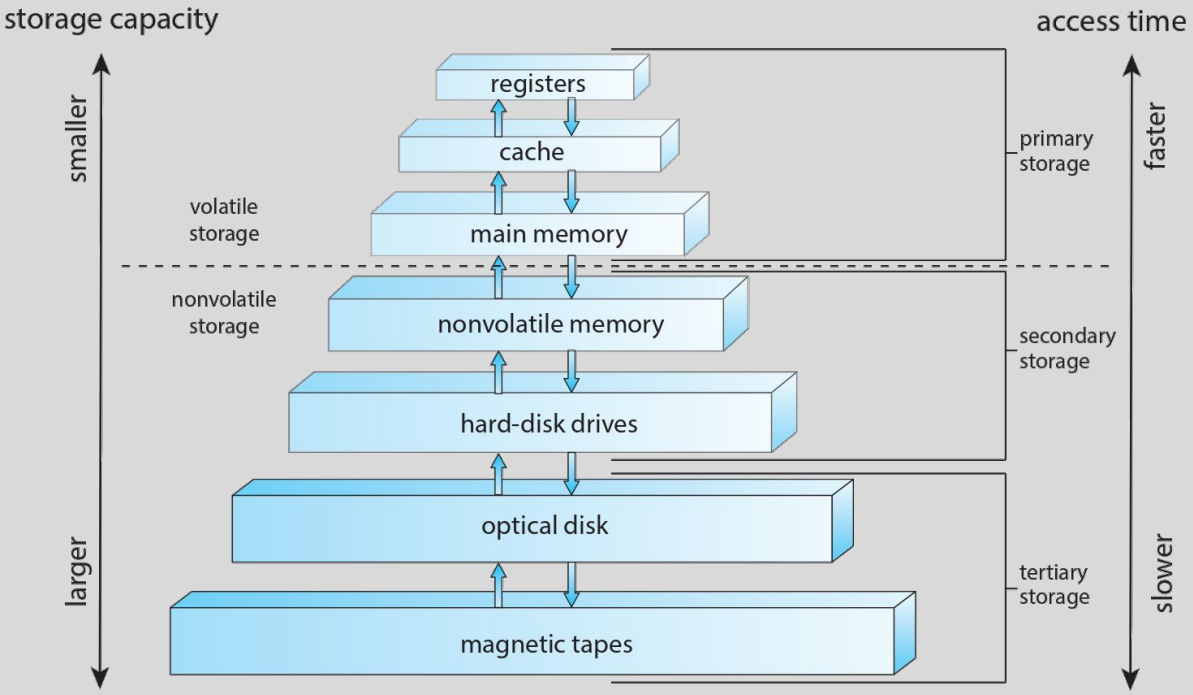
Jerarquía de almacenamiento

Regla general

- Cuanto más cerca de la CPU, más rápido y más costoso por byte.
- Cuanto más lejos, más lento y más económico por byte.

Ejemplo de jerarquía

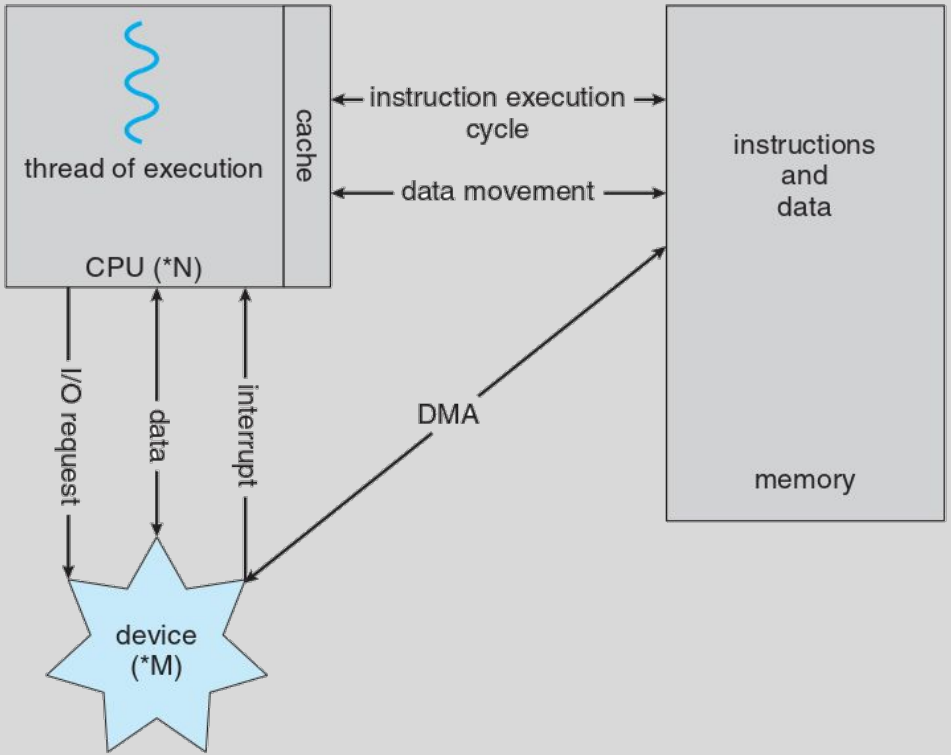
- Registros
- Caché
- Memoria principal
- Disco
- Almacenamiento terciario



E/S basada en interrupciones

E/S con interrupciones

- Es adecuada para muchas situaciones que involucran pequeñas cantidades de datos.
- Permite que la CPU no esté esperando activamente.
- Pero puede generar sobrecarga si la transferencia de datos es grande.



DMA: acceso directo a memoria

DMA

- Permite transferir bloques de datos entre dispositivo y memoria sin intervención constante de la CPU.
- La CPU inicia la operación y luego puede ocuparse de otras tareas.
- Al finalizar, el sistema es notificado.

Ventaja

- Mejora la eficiencia de las transferencias grandes.

Arquitectura del sistema

Arquitecturas posibles

- Single-processor
- Multiprocessor
- Multicore

Idea general

- La organización del hardware influye en cómo el sistema operativo administra los recursos.

Multiprocesamiento y multicore

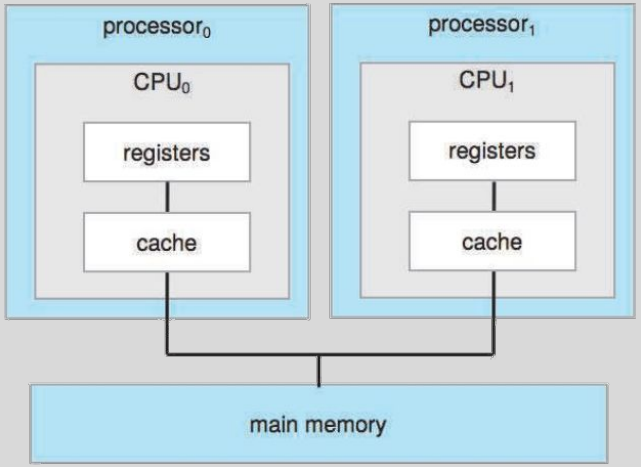
Más de una unidad de ejecución permite

- Mayor rendimiento
- Mejor disponibilidad
- Mayor capacidad de trabajo concurrente.

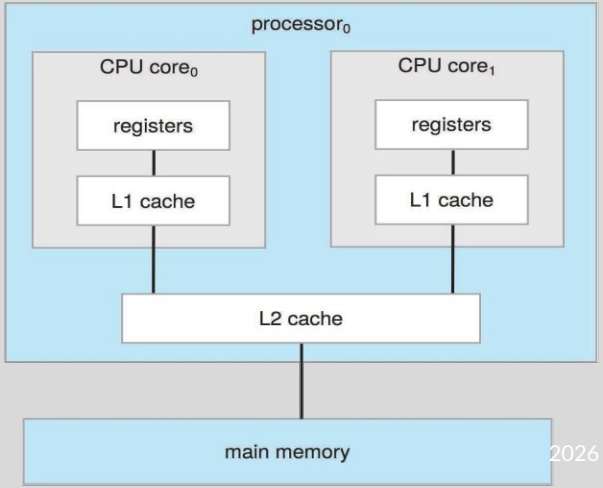
Desafío para el SO

- Coordinar el uso correcto y eficiente de esos recursos.

Multiprocessor



Multicore



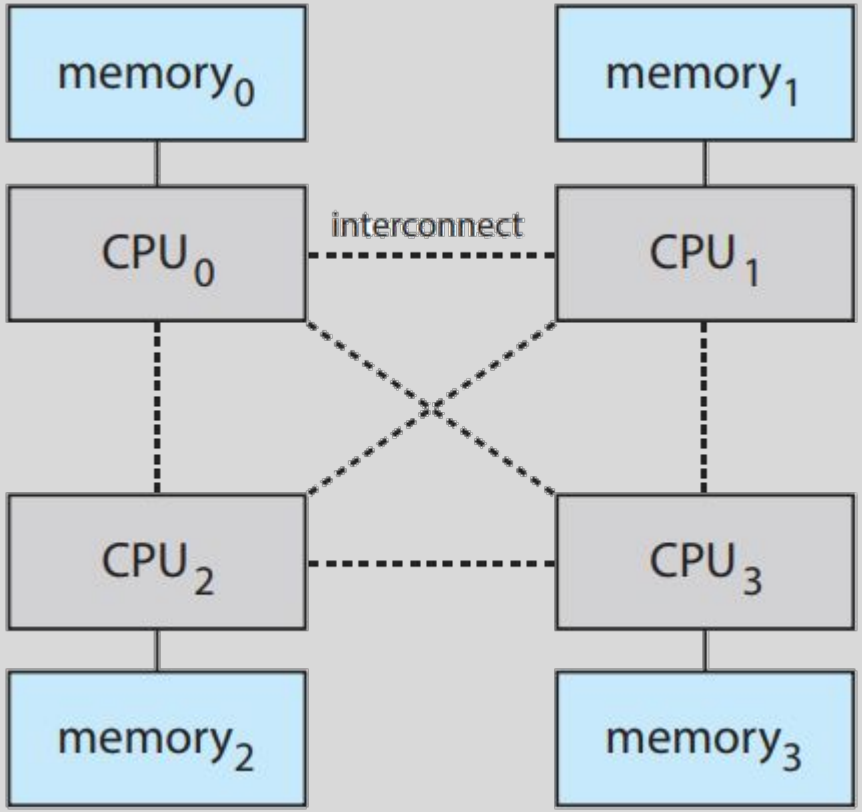
NUMA como mención

NUMA

- En algunas arquitecturas, el acceso a memoria no tiene el mismo costo para todos los procesadores.
- La cercanía entre procesador y memoria influye en el rendimiento.

Idea

- No toda memoria “cuesta lo mismo” desde cualquier CPU.



Bootstrap: cómo arranca el sistema

Secuencia de arranque

1. Se enciende la computadora.
2. El firmware inicializa el hardware (**bootstrap program**).
   a. Activa la CPU, los controladores de dispositivos y la memoria.
   b. Localiza el **kernel** del SO en disco y lo vuelca en la memoria RAM para cederle el control.
3. Inicio de Servicios y Daemons.
   a. Una vez que el kernel toma el control, inicia los **servicios esenciales** necesarios. Algunos están fuera del kernel y se conocen como **daemons** (procesos de fondo que se ejecutan continuamente para tareas específicas, como la gestión de red o de impresión).
4. El sistema operativo toma control (espera de interrupciones).
   - Cuando todo está cargado, el sistema entra en un estado de espera pasiva hasta que ocurre un **evento**, el cual se comunica mediante una **interrupción**:
     i. **De Hardware**: Por ejemplo, la señal de que el disco terminó de leer datos.
     ii. **De Software** (**System Call**): Cuando un programa de usuario solicita explícitamente un servicio al SO (abrir un archivo).

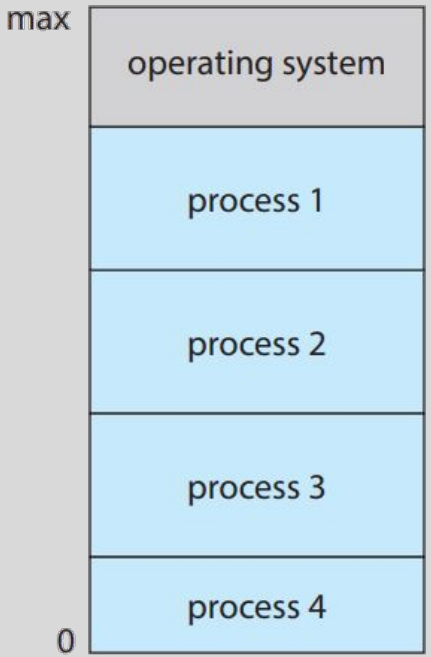
Multiprogramming

Multiprogramación

Es la capacidad el SO de ejecutar múltiples procesos. Aumenta la utilización de la CPU al organizar los programas de forma tal que la CPU siempre tenga uno para ejecutar.

Objetivo

- Busca mantener ocupada la CPU.
- Si un proceso debe esperar, el sistema puede ejecutar otro.
- Mejorar la utilización del procesador.



Multitasking

Multitarea

Es una extensión lógica de *multiprogramming*. Es la capacidad del SO para gestionar la ejecución “simultánea” de múltiples tareas o procesos. Esto se logra dividiendo el tiempo de la CPU en intervalos breves y asignándolos a cada tarea activa, alternando rápidamente entre ellas.

Resumiendo:

- Reparte el tiempo de CPU entre distintas tareas activas.
- Produce una sensación de simultaneidad.
- Mejora la respuesta percibida por el usuario.

Multiprogramming vs Multitasking

Multiprogramación

- Enfocada en aprovechar mejor la CPU
- Se apoya en alternar trabajo y espera

Multitarea

- Enfocada en compartir tiempo entre tareas activas
- Se apoya en repartir el tiempo del procesador

Ambas requieren control del sistema operativo.

El SO como gestor de recursos

El sistema operativo administra

- Procesos
- Memoria
- Archivos
- Almacenamiento
- Caché
- Entrada/Salida.

Estas áreas se profundizarán en clases posteriores.

Gestión de procesos: panorama

Un proceso necesita **recursos** (tiempo de CPU, memoria, archivos, etc) para realizar su tarea. Estos recursos se asignan al proceso mientras se ejecuta. Al finalizar, el SO recuperará los recursos reutilizables.

Sobre procesos, el SO debe ser capaz de:

- Crear y Eliminar
- Suspender y Reanudar
- Planificar ejecución
- Permitir comunicación
- Coordinar y sincronizar.

Este tema se desarrolla en la próxima clase.

Gestión de memoria: panorama

La memoria principal (RAM) funciona como un depósito enorme de bytes, cada uno con una **dirección propia**. Es el único lugar, además de los registros, desde donde la CPU puede obtener instrucciones y datos directamente.

Para maximizar la velocidad y que la CPU nunca esté ociosa, el SO mantiene varios programas en memoria simultáneamente. Esto requiere de una gestión eficiente de la memoria.

Sobre memoria, el SO debe ser capaz de:

- Saber qué parte está en uso
- Asignar y Liberar espacio
- Proteger regiones de memoria
- Mover información cuando sea necesario.

Gestión del sistema de archivos: panorama

El SO abstrae las propiedades físicas del hardware para crear el una unidad lógica de almacenamiento, el archivo. La información se guarda en diferentes medios (discos rígidos, SSD, cintas), cada uno definido por varias variables críticas:

- Velocidad de acceso: Tiempo en encontrar el dato.
- Capacidad: Espacio total de almacenamiento.
- Tasa de transferencia: Velocidad de lectura/escritura.
- Método de acceso: Si es secuencial (en orden) o aleatorio (directo).

Sobre archivos y directorios, el SO debe ser capaz de:

- Crear y Eliminar
- Organizar el acceso
- Proteger información
- Mapear estructuras lógicas sobre medios físicos

Gestión del almacenamiento masivo: panorama

Como que la memoria RAM es volátil y limitada, el almacenamiento secundario (como SSD o HDD) sirve como el depósito permanente donde residen los programas y datos, antes y después de ser procesados. Donde los dispositivos de almacenamiento actúan como:

- Origen: De donde se cargan las instrucciones hacia la memoria.
- Destino: Donde se guardan los resultados finales del procesamiento.

Sobre almacenamiento, el SO debe ser capaz de:

- Administrar espacio libre
- Decidir asignaciones
- Organizar acceso a discos
- Realizar montaje y desmontaje
- Mantener protección y consistencia

Gestión de caché y sistema de E/S: panorama

Caché

La información suele estar en discos "lentos", por eso, para trabajar rápido, el sistema copia temporalmente en lugares más veloces. El proceso es simple:

- **¿Está en la caché?** Si la respuesta es sí, se usa al instante.
- **¿No está?** Se busca en la fuente lenta, se usa y se deja una copia en la caché por si se vuelve a necesitar pronto.

Tipos de Cachés

- **de Hardware**: son automáticas y el SO no se mete con ellas (como la que ayuda a la CPU a no quedarse esperando instrucciones).
- **de Software**: En cambio, el paso de datos desde el disco a la memoria principal sí es responsabilidad directa del Sistema Operativo.

Gestión de caché y sistema de E/S: panorama

Entrada/Salida

El SO actúa como una capa transparente para ocultar todas las complicaciones técnicas de los componentes físicos (como discos o teclados), para que ni el usuario ni los programas tengan que preocuparse por ellas.

- El Controlador (**Device Controller**): es el único que sabe "hablar el idioma" específico de un dispositivo concreto (ej: los voltajes exactos para mover el cabezal de un disco).
- El **Sistema Operativo**: solo da órdenes generales ("lee este dato" o "capta esta tecla") y traduce esas señales eléctricas en algo que nosotros podamos entender.

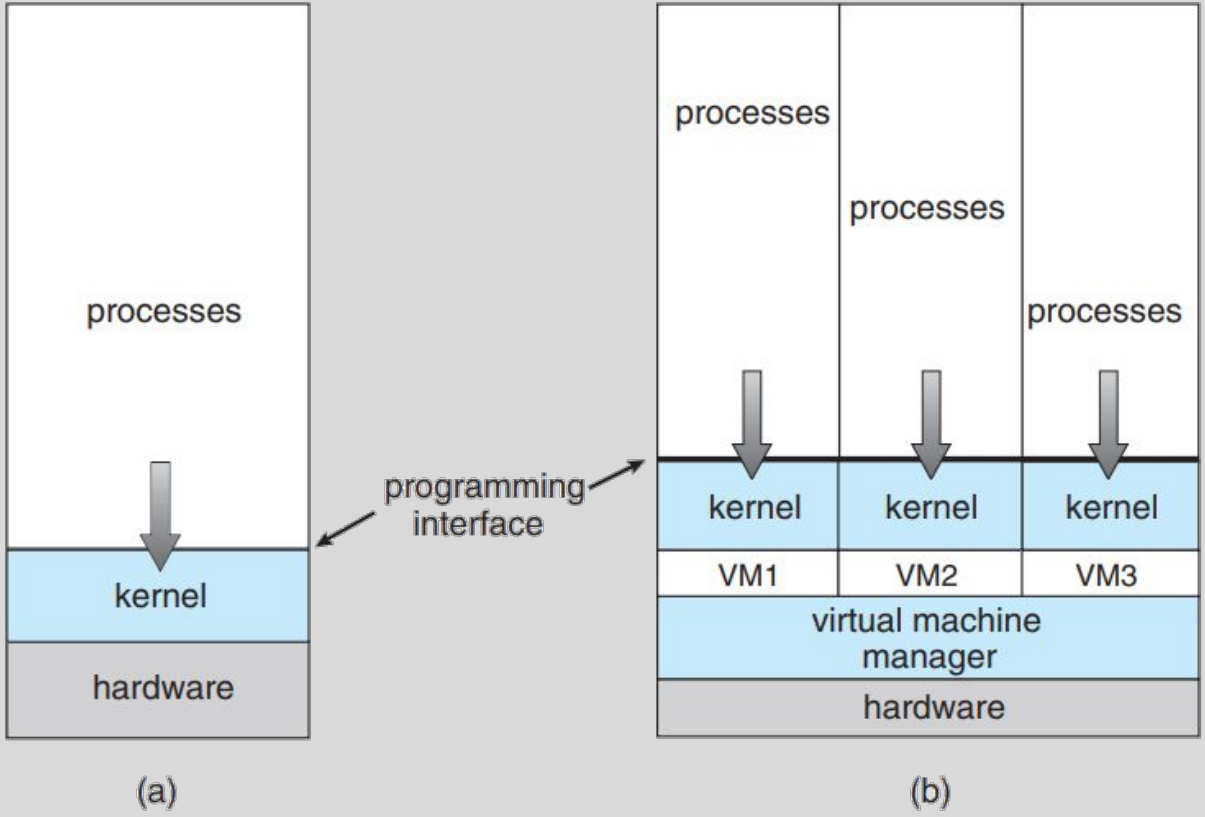
Diferentes tipos de almacenamiento

Level	1	2	3	4	5
Name	registers	cache	main memory	solid-state disk	magnetic disk
Typical size	< 1 KB	< 16MB	< 64GB	< 1 TB	< 10 TB
Implementation technology	custom memory with multiple ports CMOS	on-chip or off-chip CMOS SRAM	CMOS SRAM	flash memory	magnetic disk
Access time (ns)	0.25-0.5	0.5-25	80-250	25,000-50,000	5,000,000
Bandwidth (MB/sec)	20,000-100,000	5,000-10,000	1,000-5,000	500	20-150
Managed by	compiler	hardware	operating system	operating system	operating system
Backed by	cache	main memory	disk	disk	disk or tape

Virtualización: anticipación

Emulación vs. Virtualización

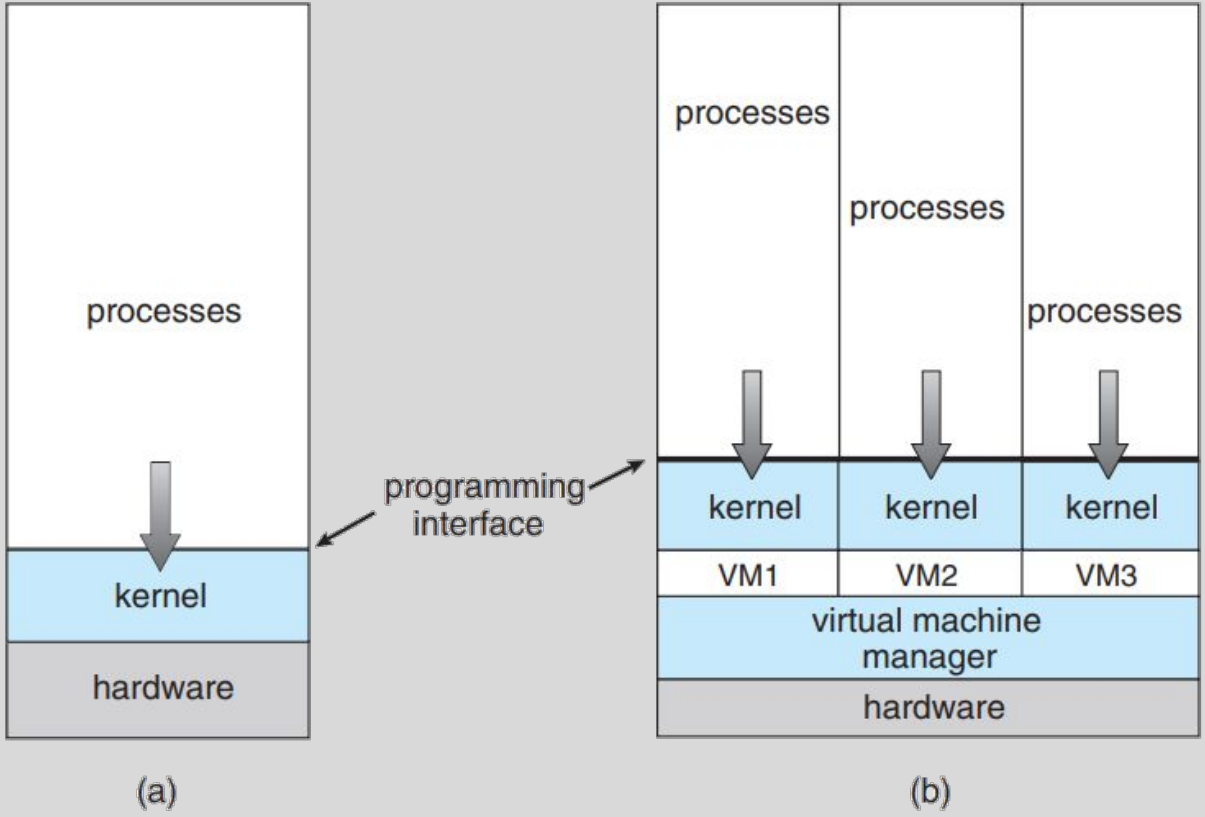
- Emulación (Simular): Consiste en imitar el hardware de una computadora mediante software.
  Como cada instrucción del sistema original debe "traducirse" al sistema nuevo, es un proceso lento. Esto es posible gracias al principio de equivalencia: cualquier cosa que haga el hardware, puede recrearse con código.



Virtualización: anticipación

- Virtualización (Compartir): Acá no hay traducción. El sistema corre sobre la misma arquitectura de CPU para la que fue diseñado. Lo que hace el software es repartir los componentes físicos (CPU, RAM, disco) para que parezca que cada sistema operativo tiene su propia PC independiente.

Este tema se verá más adelante con mayor detalle.



Sistemas distribuidos: anticipación

Sistemas distribuidos

Un sistema distribuido es un conjunto de sistemas físicamente separados, que se conectan en red para proporcionar a los usuarios acceso a los recursos que mantiene.

- Integran múltiples computadoras conectadas.
- Permiten compartir recursos y ofrecer servicios coordinados.
- Son claves para escalabilidad, disponibilidad y trabajo en red.

Cierre general

Idea final de la clase, un resumen de lo que vimos hoy.

El sistema operativo:

- Abstrae el hardware
- Administra recursos
- Protege el sistema
- Coordina la ejecución
- Y crea el entorno donde viven los programas

En la próxima clase

Próxima clase: Procesos

Vamos a ver:

- Definición formal de proceso
- Estados
- PCB (*Process Control Block*)
- Cambio de contexto
- Y la base para hilos, planificación y sincronización

Muchas Gracias

Jeremías Fassi

Javier E. Kinter

2026